

Lab 25 - Troubleshooting Triage: Application, Node and Control Plane

Field	Value
Lab id	Lab 25
Day / module	Day 5 - Troubleshooting
Duration	55 minutes
Namespace	kcna-lab25
Maps to	LO6 Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework K7 Interactions among various IT components
Cluster	Single-node kind, Kubernetes v1.30+ (built and validated against v1.37.0)

How to read the expected-output blocks. Pod name suffixes, IPs, AGE

columns and timestamps vary at runtime. Match the STATUS strings, the

event reasons and the error messages - those are the diagnostic

signal and they are stable.

Safety statement - read before Part 6

The control-plane section of this lab is strictly read-only. You will

inspect static Pod manifests, read kube-system logs and query health

endpoints. You will never be asked to modify, restart, scale or delete

anything in kube-system, and you must not do so. On a shared training

cluster, breaking the control plane ends the day for everyone. Control-plane

failure modes are discussed, not induced.

1. Objective

By the end of this lab you will be able to:

1. Apply a repeatable five-question triage method that identifies the

phase a workload failed in before you form any hypothesis about why.

2. Diagnose four distinct real failures - image pull, CrashLoopBackOff,

Pending, and a Service with no endpoints - from cluster evidence alone.

3. Choose the right tool per phase: kubectl get for phase,

kubectl describe/kubectl events for scheduler and kubelet decisions, kubectl logs [--previous] for application behaviour, and EndpointSlice inspection for routing.

4. Recognise container exit codes - 0, 1, 127, 137 - and what each implies.
5. Inspect the control plane read-only with `--raw='/readyz?verbose'`, static Pod manifests and kube-system logs, and describe common control-plane failure modes without inducing them.
6. Reuse a written decision tree in the practical assessment.

2. Prerequisites

- * A running single-node kind cluster.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

- * You are in the lab directory:

```
cd courseware/labs/lab-25-troubleshooting-triage
```

- * Images used, all pinned: nginx:1.27-alpine, busybox:1.36, alpine:3.20, hashicorp/http-echo:1.0.

- * Labs 22-24 are soft prerequisites: probes, events, logs.

- * Some Part 6 commands use docker exec against the kind node container. Get your node's name first - it is used throughout:

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	2d	v1.37.0

```
NODE=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "node = $NODE"
```

```
node = kind-control-plane
```

3. Scenario

It is the Monday after the Meridian Freight change freeze lifted. Over the weekend four teams pushed manifests into the staging depot cluster and went home. The Depot Platform Squad's on-call queue has four tickets waiting.

Read them:

```
column -s, -t data/incident-queue.csv
```

ticket	raised_by	priority	workload	manifest_to_apply
DEP-1041	gate-ops-sg-tuas	P2	depot-api	broken/01-depot-api-imagepull.yaml
	sla_minutes			
	10			
DEP-1042	platform-oncall	P1	shipment-worker	broken/02-shipment-worker-crashloop.yaml
	Worker keeps restarting. It prints one line then vanishes. Restart count is climbing fast.			
	10			
DEP-1043	platform-oncall	P2	manifest-indexer	broken/03-manifest-indexer-pending.yaml
	Applied 20 minutes ago and nothing has happened. kubectl logs says there is no container.			
	10			
DEP-1044	driver-app-team	P1	depot-tracker	broken/04-depot-tracker-no-endpoints.yaml
	Pods are green, dashboard is green, but every request to the tracker service times out.			
	15			

Every reporter has described a symptom, and every one of them has already guessed at a cause. None of the guesses is reliable. Your job is to work the method, not the guesses.

These four manifests are schema-valid. kubectl apply will accept all of them without complaint. That is deliberate and it is the most important thing about this lab: passing validation is not the same as working. The API server checks structure; it cannot check that an image tag is spelled correctly, that a binary exists, that a node has 512Gi of RAM, or that a selector matches anything.

4. The triage method

Before you touch the cluster, learn the five questions. Ask them in order and stop at the first NO - that is your failure phase.

[illegible]

▼

Q1 Is it SCHEDULED? `kubectl get pod -o wide` → NODE column

- NO → STATUS Pending. The SCHEDULER refused.

- No node, no containers, NO LOGS EXIST.

- `kubectl describe pod` → Warning FailedScheduling

- Insufficient cpu/memory | didn't match node affinity |

- untolerated taint | no available PV

▼ YES

Q2 Did the image PULL? `kubectl describe pod` → events

- NO → ErrImagePull / ImagePullBackOff.

- Still no application logs - nothing has run.

- wrong name/tag | private registry, no imagePullSecret |

```

- registry unreachable | process degraded,
+ registry unreachable | rate limited

```

▼ YES

Q3 Did the container START and STAY UP?

- `kubectl get pod → RESTARTS, STATUS`

- NO → CrashLoopBackOff / Error / OOMKilled.

■ NOW logs exist. Use --previous.

- `kubectl logs --previous + exit code`

```

127 command not found | 1 app error |

```

```

137 SIGKILL (OOM or liveness) | 0 exited cleanly

```

▼ YES

Q4 Is it READY? `kubectl get pod` → READY n/n

```

10 10 READY:             kubelet got pod "READY" N/A
■ NO → readiness probe failing. Container runs but is withdrawn

```

- from Service backends. (Lab 22)

```

→ kubectl describe pod → Warning Unhealthy

```

▼ YES

Q5 Does the Service have ENDPOINTS?

```
■          kubectl get endpointslices -l kubernetes.io/service-name=<svc>
```

- NO → SELECTOR MISMATCH, or no Ready Pods, or wrong port name.

```

■ SILENT: no events, no restarts, no logs.

```

▼ YES

Pods and routing are healthy. Now look OUTSIDE the workload:

NetworkPolicy | DNS | Ingress | the application's own config.

Why the order matters. Each question is only meaningful if the previous

one answered YES. Asking for logs on a Pending Pod is not just useless, it is

actively misleading - it tells you "no container", which sounds like a

container problem and is not. Phase first, cause second.

— — —

5. Step-by-step procedure

Part 1 - Namespace and the known-good baseline

Step 1.1 - Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab25 created
```

Step 1.2 - Apply the healthy reference workload. Keep it running all lab: it is your control sample.

```
kubectl apply -f manifests/10-depot-portal-healthy.yaml
kubectl rollout status deployment/depot-portal -n kcna-lab25 --timeout=120s
```

```
deployment.apps/depot-portal created
service/depot-portal created
pod/triage-client created
deployment "depot-portal" successfully rolled out
```

Step 1.3 - Record what "working" looks like. All five questions answer YES.

```
kubectl get pods -n kcna-lab25 -o wide -l app=depot-portal
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-portal
```

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
depot-portal-6f9c8d7b54-4nvqz	1/1	Running	0	40s	10.244.0.21	kind-control-plane	
depot-portal-6f9c8d7b54-mzt8h	1/1	Running	0	40s	10.244.0.22	kind-control-plane	

NAME	ADDRESS TYPE	PORTS	ENDPOINTS	AGE
depot-portal-4bp7q	IPv4	80	10.244.0.21,10.244.0.22	40s

Step 1.4 - Confirm it actually serves traffic.

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -qO- --timeout=5 http://depot-portal.kcna-lab25.svc.cluster.local/ | head -4
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Part 2 - Apply all four broken manifests

Step 2.1 - Apply them together. Note that all four are accepted.

```
kubectl apply -f broken/
```

```
deployment.apps/depot-api created
deployment.apps/shipment-worker created
deployment.apps/manifest-indexer created
deployment.apps/depot-tracker created
service/depot-tracker created
```

Five objects created, zero errors. Nothing here was rejected by

validation. Every one of these failures is a *runtime* failure.

Step 2.2 - Wait ~90 seconds, then take one snapshot. This single command is your triage starting point for all four tickets.

```
kubectl get pods -n kcna-lab25 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS GATES		
depot-api-7d4b9c6f85-x2ktp	0/1	ImagePullBackOff	0	92s	10.244.0.23
kind-control-plane	<none>	<none>			
depot-portal-6f9c8d7b54-4nvqz	1/1	Running	0	4m	10.244.0.21
kind-control-plane	<none>	<none>			
depot-portal-6f9c8d7b54-mzt8h	1/1	Running	0	4m	10.244.0.22
kind-control-plane	<none>	<none>			
depot-tracker-5c8f7b9d64-h6wnr	1/1	Running	0	92s	10.244.0.25
kind-control-plane	<none>	<none>			
depot-tracker-5c8f7b9d64-q4jsx	1/1	Running	0	92s	10.244.0.26
kind-control-plane	<none>	<none>			
manifest-indexer-8b7d5c9f47-lm3zq	0/1	Pending	0	92s	<none>
<none>	<none>	<none>			
shipment-worker-59d4c8b7f6-w8kdt	0/1	CrashLoopBackOff	4 (21s ago)	92s	10.244.0.24
kind-control-plane	<none>	<none>			
triage-client	1/1	Running	0	4m	10.244.0.20
kind-control-plane	<none>	<none>			

Step 2.3 - Read the snapshot with Q1 in mind, before diagnosing anything. Fill in the q1_scheduled column of data/triage-worksheet.csv now.

Pod	NODE	Q1 scheduled?	Earliest possible failure phase
depot-api	kind-control-plane	YES	Q2 or later
manifest-indexer	<none>	NO	Q1 - scheduling
shipment-worker	kind-control-plane	YES	Q3 or later
depot-tracker	kind-control-plane	YES	Q4 or later - and both are 1/1 Running, so Q5

Notice you have already localised all four failures to different phases, from

one `kubectl get pods -o wide`, without a single `describe` or `logs`.

Part 3 - DEP-1041: depot-api (image pull)

Step 3.1 - Q1: scheduled? Yes - it has a node and an IP. Move to Q2.

Step 3.2 - Q2: did the image pull? describe is where the kubelet reports.

```
kubectl describe pod -n kcna-lab25 -l app=depot-api | tail -12
```

Events:

Type	Reason	Age	From	Message
Normal	Scheduled	2m12s	default-scheduler	Successfully assigned kcna-lab25/depot-api-7d4b9c6f85-x2ktp to kind-control-plane
Normal	Pulling	47s (x4 over 2m11s)	kubelet	Pulling image "nginx:1.27-alpne"
Warning	Failed	45s (x4 over 2m9s)	kubelet	Failed to pull image "nginx:1.27-alpne": failed to pull and unpack image "docker.io/library/nginx:1.27-alpne": failed to resolve reference "docker.io/library/nginx:1.27-alpne": docker.io/library/nginx:1.27-alpne: not found
Warning	Failed	45s (x4 over 2m9s)	kubelet	Error: ErrImagePull
Normal	BackOff	9s (x7 over 2m8s)	kubelet	Back-off pulling image "nginx:1.27-alpne"
Warning	Failed	9s (x7 over 2m8s)	kubelet	Error: ImagePullBackOff

Step 3.3 - Confirm that logs are useless here, and understand why.

```
kubectl logs -n kcna-lab25 -l app=depot-api
```

Error from server (BadRequest): container "api" in pod "depot-api-7d4b9c6f85-x2ktp" is waiting to start: trying and failing to pull image

No image means no container means no logs. Reporters who say "there are no logs" often think that is the problem. It is a *consequence* of the phase.

Step 3.4 - Extract the exact image reference from the API.

```
kubectl get deploy depot-api -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
nginx:1.27-alpne
```

Step 3.5 - Compare against the known-good baseline. The diff *is* the bug.

```
kubectl get deploy depot-portal -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
nginx:1.27-alpine
```

alpne versus alpine. Root cause: misspelt image tag.

Distinguish the three image-pull failure signatures, because the fixes are completely different:

lastError / message	Meaning	Fix
--- --- ---	not found / manifest unknown	Name or tag is wrong Correct the reference
unauthorized / authentication required	Private registry, no credentials	Create an imagePullSecret and reference it
no such host / i/o timeout	Registry unreachable from the node	Network, DNS, proxy or firewall
toomanyrequests	Docker Hub rate limit	Authenticate, or use a mirror/pull-through cache

Step 3.6 - Record DEP-1041 in data/triage-worksheet.csv: first_failing_question = q2_pulled.

Part 4 - DEP-1042 and DEP-1043

DEP-1042: shipment-worker (CrashLoopBackOff)

Step 4.1 - Q1 yes, Q2 yes (it has restarted, so the image pulled). Q3 is where it fails. Confirm from events.

```
kubectl describe pod -n kcna-lab25 -l app=shipment-worker | tail -10
```

```

Events:
  Type    Reason      Age           From              Message
  ----    -
  Normal  Scheduled   3m8s          default-scheduler Successfully assigned kcna-
lab25/shipment-worker-59d4c8b7f6-w8kdt to kind-control-plane
  Normal  Pulled      94s (x5 over 3m7s)  kubelet           Container image "busybox:1.36"
already present on machine
  Normal  Created     94s (x5 over 3m7s)  kubelet           Created container: worker
  Normal  Started     94s (x5 over 3m7s)  kubelet           Started container worker
  Warning BackOff      12s (x14 over 3m4s)  kubelet           Back-off restarting failed
container worker in pod shipment-worker-59d4c8b7f6-w8kdt_kcna-lab25(...)

```

Pulled, Created and Started are all Normal. The image was fine and

the container genuinely ran. This is unambiguously a Q3 failure, not Q2.

Contrast with Step 3.2 where Pulling was followed by Failed.

Step 4.2 - Now logs are meaningful. Use --previous, because the current container is in back-off.

```
kubectl logs -n kcna-lab25 -l app=shipment-worker --previous --tail=10
```

```
[worker] starting shipment worker for depot sg-tuas
/bin/sh: /usr/local/bin/depot-shipment-worker: not found
```

Step 4.3 - Confirm with the exit code, which is authoritative.

```

WORKER=$(kubectl get pods -n kcna-lab25 -l app=shipment-worker \
-o jsonpath='{.items[0].metadata.name}')
kubectl get pod -n kcna-lab25 "$WORKER" -o jsonpath='{range .status.containerStatuses[*]}{.
name}{": " lastState.exitCode="}{.lastState.terminated.exitCode}{": " reason="}{.lastState.
terminated.reason}{": "}{end}'

```

```
worker lastState.exitCode=127 reason=Error
```

Exit code 127 is the shell's "command not found". Learn the four you will

meet constantly:

Code	Meaning	Typical cause
0	Clean exit	Job finished; wrong restartPolicy for a long-running workload
1	Application error	Bad config, missing env var, failed dependency - read the log
127	Command not found	command/args disagree with the image contents
137	SIGKILL (128+9)	OOMKilled, or a failed liveness probe. Check reason

Step 4.4 - Prove the binary genuinely is not in that image. You cannot exec into the crash-looping container - it is never up long enough - so run a throwaway Pod from the *same* image and look for yourself. Note --rm, which removes it immediately afterwards.

```
kubectl run image-probe --rm -i --restart=Never \
--image=busybox:1.36 -n kcna-lab25 -- ls /usr/local/bin/
```

```
pod "image-probe" deleted
```

Empty listing, then the Pod is gone. Confirm the shell agrees:

```
kubectl run image-probe --rm -i --restart=Never \
  --image=busybox:1.36 -n kcna-lab25 -- \
  sh -c 'command -v depot-shipment-worker || echo "NOT IN IMAGE"'
```

```
NOT IN IMAGE
pod "image-probe" deleted
```

Root cause: the container command references a binary the image does not contain.

Step 4.5 - Record DEP-1042: first_failing_question = q3_started, exit code 127.

DEP-1043: manifest-indexer (Pending)

Step 4.6 - Q1 fails immediately: STATUS Pending, NODE <none>. Confirm that logs cannot help, so you do not waste SLA time there.

```
kubectl logs -n kcna-lab25 -l app=manifest-indexer
```

```
Error from server (BadRequest): pod manifest-indexer-8b7d5c9f47-lm3zq does not have a host assigned
```

Read that error precisely. does not have a host assigned is the

unscheduled-Pod signature, and it is *different* from the image-pull Pod's

waiting to start: trying and failing to pull image in Step 3.3. Two Pods,

both with no logs, two completely different phases. The error text tells you

which.

Step 4.7 - Go straight to the scheduler's verdict.

```
kubectl describe pod -n kcna-lab25 -l app=manifest-indexer | tail -8
```

```
Events:
  Type            Reason              Age             From              Message
  ----            -
  Warning         FailedScheduling    4m5s (x2 over 4m20s)  default-scheduler  0/1 nodes are available:
1 Insufficient cpu, 1 Insufficient memory. preemption: 0/1 nodes are available: 1 No
preemption victims found for incoming pod.
```

Read 0/1 nodes are available literally. It is a per-node tally of

reasons: Insufficient cpu, Insufficient memory, `node(s) had untolerated

taint, node(s) didn't match Pod's node affinity/selector, node(s) had

volume node affinity conflict`. On a big cluster you get a breakdown per

reason, and the tally tells you how much of the cluster each predicate ruled

out.

Step 4.8 - Compare demand against supply. This is the arithmetic that settles it.

```
kubectl get deploy manifest-indexer -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].resources.requests}{"\n"}'
kubectl get node "$NODE" -o jsonpath='{.status.allocatable.cpu}{ " cpu / "}{.status.allocatable.
memory}{ " memory\n"}'
```

```
{"cpu": "64", "memory": "512Gi"}
10 cpu / 8039384Ki memory
```

64 cores and 512Gi requested; the node can offer 10 cores and roughly 7.7Gi. Root cause: resource requests exceed any node's allocatable capacity.

****The exact allocatable figures above depend on your machine and your Docker**

Desktop resource allocation** - yours will differ. The *relationship* is what

matters: requested ☐ allocatable.

Step 4.9 - See the alternative Pending cause without applying it. Open broken/03-manifest-indexer-pending.yaml and read the commented-out nodeAffinity block. Uncommenting it (with sane resources) produces the same Pending status but a different message:

```
0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector.
```

Same symptom, different root cause, different fix. This is precisely why

the method separates *phase* from *cause*. Pending is a phase; there are at

least five distinct causes.

Step 4.10 - Record DEP-1043: first_failing_question = q1_scheduled.

Part 5 - DEP-1044: depot-tracker (Service with no endpoints)

This is the hardest of the four, because nothing looks wrong.

Step 5.1 - Q1-Q4 all pass. Verify that for yourself:

```
kubectl get pods -n kcna-lab25 -l app=depot-tracker -o wide
```

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
depot-tracker-5c8f7b9d64-h6wnr		1/1	Running	0	6m20s	10.244.0.25	kind-
control-plane	<none>	<none>					
depot-tracker-5c8f7b9d64-q4jsx		1/1	Running	0	6m20s	10.244.0.26	kind-
control-plane	<none>	<none>					

Scheduled, pulled, started, Ready 1/1, zero restarts. A status dashboard would be entirely green - exactly as the driver-app team reported.

Step 5.2 - Reproduce the user-facing symptom.

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -q0- --timeout=5 http://depot-tracker.kcna-lab25.svc.cluster.local/
```

```
wget: can't connect to remote host (10.96.184.22): Connection refused
command terminated with exit code 1
```

Connection refused from a ClusterIP with no backends. DNS resolved

(there is an IP), so this is not a DNS fault. Note the contrast: a

NetworkPolicy block or a missing route usually gives a timeout, whereas an

endpoint-less Service typically gives an immediate refusal. That

distinction is worth real minutes during an incident.

Step 5.3 - Q5. Ask the routing layer directly.

```
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-tracker
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-tracker-9xz4k	IPv4	<unset>	<unset>	6m45s

ENDPOINTS <unset>. Compare with the healthy baseline from Step 1.3, which listed two addresses. This is the answer.

Step 5.4 - Find out *why* there are no endpoints. There are three possible causes; eliminate them in order.

Cause A - no Ready Pods? Already disproved in Step 5.1.

Cause B - selector mismatch. Compare the two label sets:

```
echo "Service selects:"
kubectl get svc depot-tracker -n kcna-lab25 -o jsonpath='{.spec.selector}{"\n"}'
echo "Pods are labelled:"
kubectl get pods -n kcna-lab25 -l app=depot-tracker \
-o jsonpath='{.items[0].metadata.labels}{"\n"}'
```

```
Service selects:
{"app":"depot-tracking"}
Pods are labelled:
{"app":"depot-tracker", "app.kubernetes.io/part-of":"depot-portal", "pod-template-hash":
"5c8f7b9d64"}
```

depot-track**ing** versus depot-track**er**. Root cause found.

Step 5.5 - Confirm decisively by asking the API to select with the Service's own selector. Zero results proves the selector matches nothing.

```
kubectl get pods -n kcna-lab25 -l app=depot-tracking
```

No resources found in kcna-lab25 namespace.

Commit this one command to memory. `kubectl get pods -l <the service's

selector>` answers "does this Service select anything?" in one step, and it

works no matter how complex the selector is.

Step 5.6 - Cause C, for completeness: a port name mismatch. If Service.spec.ports[].targetPort names a port the container does not declare, you get endpoints but with PORTS <unset> and traffic still fails. Check the healthy service to see what a correct mapping looks like:

```
kubectl get svc depot-portal -n kcna-lab25 -o jsonpath='{.spec.ports}{"\n"}'
kubectl get deploy depot-portal -n kcna-lab25 \
-o jsonpath='{.spec.template.spec.containers[0].ports}{"\n"}'
```

```
[{"name": "http", "port": 80, "protocol": "TCP", "targetPort": "http"}]
[{"containerPort": 80, "name": "http", "protocol": "TCP"}]
```

targetPort: "http" resolves because the container declares a port named http.

Step 5.7 - Record DEP-1044: first_failing_question = q5_endpoints.

Part 6 - Node and control-plane inspection (READ-ONLY)

Nothing in this Part modifies anything. Every command reads. Do not

substitute a delete, edit, patch, scale or restart for any of them.

Step 6.1 - Node health. Ready is one of several conditions; the pressure conditions are what tell you a node is about to start evicting Pods.

```
kubectl get nodes -o wide
kubectl describe node "$NODE" | sed -n '/^Conditions:/,/^\Addresses:/p'
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-																									
kind-control-plane	Ready	control-plane	2d	v1.37.0	172.18.0.2	<none>	Debian GNU/Linux 12 (bookworm)																									
Conditions: <table border="1"> <thead> <tr> <th>Type</th> <th>Status</th> <th>LastHeartbeatTime</th> <th>LastTransitionTime</th> <th>Reason</th> </tr> </thead> <tbody> <tr> <td>MemoryPressure</td> <td>False</td> <td>...</td> <td>...</td> <td>KubeletHasSufficientMemory</td> </tr> <tr> <td>DiskPressure</td> <td>False</td> <td>...</td> <td>...</td> <td>KubeletHasNoDiskPressure</td> </tr> <tr> <td>PIDPressure</td> <td>False</td> <td>...</td> <td>...</td> <td>KubeletHasSufficientPID</td> </tr> <tr> <td>Ready</td> <td>True</td> <td>...</td> <td>...</td> <td>KubeletReady</td> </tr> </tbody> </table>								Type	Status	LastHeartbeatTime	LastTransitionTime	Reason	MemoryPressure	False	...	...	KubeletHasSufficientMemory	DiskPressure	False	...	...	KubeletHasNoDiskPressure	PIDPressure	False	...	...	KubeletHasSufficientPID	Ready	True	...	...	KubeletReady
Type	Status	LastHeartbeatTime	LastTransitionTime	Reason																												
MemoryPressure	False	...	...	KubeletHasSufficientMemory																												
DiskPressure	False	...	...	KubeletHasNoDiskPressure																												
PIDPressure	False	...	...	KubeletHasSufficientPID																												
Ready	True	...	...	KubeletReady																												

Node failure modes to recognise, not to induce.

* Ready False / Unknown - the kubelet has stopped posting status. After

--node-monitor-grace-period the node controller taints the node and Pods

are evicted. Unknown usually means the node or its network is gone.

* DiskPressure True - the kubelet starts evicting Pods and garbage

collecting images. Very often it is the container log directory or unused

images filling the disk.

* MemoryPressure True - eviction by QoS class: BestEffort first, then

Burstable, Guaranteed last. This is the practical reason to set

requests and limits.

Step 6.2 - Node capacity versus commitment. This is what settles "why can nothing schedule?" questions like DEP-1043.

```
kubectl describe node "$NODE" | sed -n '/Allocated resources/,/^Events/p'
```

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests          Limits
-----
cpu                 1050m (10%)      1300m (13%)
memory              690Mi (8%)       1224Mi (15%)
ephemeral-storage   0 (0%)           0 (0%)
```

The scheduler makes decisions on REQUESTS, not on live usage. A node can be 5% busy and still reject a Pod because its requests are fully committed. This is the single most common source of "there's plenty of free memory, why won't it schedule?".

Step 6.3 - Control-plane health via the API server's own endpoints. This is the modern, supported check.

```
kubectl get --raw='/readyz?verbose' | tail -20
```

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
[+]poststarthook/start-apiserver-admission-initializer ok
[+]poststarthook/generic-apiserver-start-informers ok
[+]poststarthook/start-kube-apiserver-identity-lease-controller ok
[+]poststarthook/bootstrap-controller ok
[+]poststarthook/rbac/bootstrap-roles ok
[+]poststarthook/scheduling/bootstrap-system-priority-classes ok
[+]poststarthook/start-cluster-authentication-info-controller ok
[+]shutdown ok
readyz check passed
```

```
kubectl get --raw='/livez?verbose' | tail -5
```

```
[+]etcd ok
[+]autoregister-completion ok
[+]shutdown ok
livez check passed
```

readyz vs livez vs healthz. livez answers "is the process functional?"; readyz answers "should it receive traffic?" - the same distinction as container probes in Lab 22. healthz is the deprecated combined endpoint. [-]etcd failed in this listing is the single most important line to recognise: it means the API server cannot reach its datastore, and essentially everything else in the cluster is about to fail.

Step 6.4 - The deprecated check, for recognition only.

```
kubectl get componentstatuses
```

```
Warning: v1 ComponentStatus is deprecated in v1.19+
NAME                STATUS    MESSAGE   ERROR
scheduler            Healthy   ok
controller-manager   Healthy   ok
etcd-0               Healthy   ok
```

Expect this to be unhelpful or absent. ComponentStatus was deprecated in

v1.19 and is not reliable on modern clusters - it may print an empty list, or

the API may be gone entirely. If it errors, that is not a fault. Use

/readyz?verbose instead. You should recognise componentstatuses in older

documentation and know why not to depend on it.

Step 6.5 - Control-plane Pods. On a kubeadm/kind cluster these are static Pods, managed directly by the kubelet from files on disk - not by any controller.

```
kubectl -n kube-system get pods -o wide \
-l tier=control-plane
```

NAME NODE	READY	STATUS	RESTARTS	AGE	IP
etcd-kind-control-plane	1/1	Running	0	2d	172.18.0.2
kind-control-plane					
kube-apiserver-kind-control-plane	1/1	Running	0	2d	172.18.0.2
kind-control-plane					
kube-controller-manager-kind-control-plane	1/1	Running	1	2d	172.18.0.2
kind-control-plane					
kube-scheduler-kind-control-plane	1/1	Running	1	2d	172.18.0.2
kind-control-plane					

Step 6.6 - Read the static Pod manifests on the node. Read-only - cat, never an editor.

```
docker exec "$NODE" ls -l /etc/kubernetes/manifests/
```

```
total 16
-rw----- 1 root root 2405 Sep  3 08:11 etcd.yaml
-rw----- 1 root root 3896 Sep  3 08:11 kube-apiserver.yaml
-rw----- 1 root root 3428 Sep  3 08:11 kube-controller-manager.yaml
-rw----- 1 root root 1463 Sep  3 08:11 kube-scheduler.yaml
```

```
docker exec "$NODE" grep -E '^s+ --(etcd-servers|secure-port|advertise-address|service-
cluster-ip-range)' \
/etc/kubernetes/manifests/kube-apiserver.yaml
```

```
- --advertise-address=172.18.0.2
- --etcd-servers=https://127.0.0.1:2379
- --secure-port=6443
- --service-cluster-ip-range=10.96.0.0/16
```

The static Pod mechanism, and its danger. The kubelet watches

/etc/kubernetes/manifests/ and runs whatever it finds there. There is no

Deployment, no ReplicaSet and no API-server involvement - which is exactly how the API server itself can be started. The consequence: **editing a file in this directory restarts a control-plane component immediately**, with no rollout, no validation and no undo. A single YAML typo in kube-apiserver.yaml takes the cluster's API offline, and you then have no kubectl with which to fix it. That is why this lab only reads.

Step 6.7 - Control-plane logs, read-only.

```
kubectl -n kube-system logs kube-scheduler-"$NODE" --tail=8
```

```
I0905 06:44:02.118472      1 schedule_one.go:...] "Successfully bound pod to node" pod="kcna-lab25/depot-portal-6f9c8d7b54-4nvqz" node="kind-control-plane"
I0905 06:45:31.884210      1 schedule_one.go:...] "Unable to schedule pod; no fit; waiting"
pod="kcna-lab25/manifest-indexer-8b7d5c9f47-lm3zq" err="0/1 nodes are available: 1
Insufficient cpu, 1 Insufficient memory."
```

Your DEP-1043 failure, from the scheduler's own perspective. The Event you read in Step 4.7 and this log line are the same decision reported through two different channels. When Events have expired (Lab 23: one-hour TTL), the component log is where the evidence still lives.

Step 6.8 - If a control-plane static Pod is not running at all, kubectl cannot show you its logs - so read the container runtime directly on the node. Know this command *before* you need it.

```
docker exec "$NODE" crictl ps --name kube-apiserver
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID
POD						
3f8a1c2e9b7d	a1b2c3d4e5f6	2 days ago	Running	kube-apiserver	0	9c8b7a6d5e4f
kube-apiserver-kind-control-plane						

Discussed, not induced - the control-plane failure modes.

| Component down | What still works | What breaks | First check |

|---|---|---|

| kube-apiserver | Running Pods keep running; kube-proxy keeps routing | All kubectl; every controller; all cluster state changes | crictl ps on the node; /etc/kubernetes/manifests/kube-apiserver.yaml; etcd reachability |

| etcd | Nothing meaningful - the API server goes read-only then fails | Everything | kubectl get --raw='/readyz?verbose' → [-]etcd failed |

| kube-scheduler | Existing Pods run normally | New Pods stay Pending forever with no FailedScheduling event at all | kubectl -n kube-system logs kube-scheduler-<node> |

| kube-controller-manager | Existing Pods run | Deployments do not create ReplicaSets; failed nodes are never drained; endpoints stop updating | kubectl -n kube-system logs kube-controller-manager-<node> |

| kubelet (on one node) | Other nodes fine | That node goes NotReady; its Pods are evicted after the grace period | docker exec <node> systemctl status kubelet |

>

****The diagnostic that distinguishes "scheduler down" from "unschedulable**

Pod": a Pending Pod with no FailedScheduling event whatsoever means**

nothing is even evaluating it - suspect the scheduler. A Pending Pod with

a FailedScheduling event means the scheduler is alive and is telling you

exactly why it refused, as in DEP-1043.

Part 7 - Apply the fixes and verify

Step 7.1 - Only now, with all four root causes written into your worksheet, apply the fixes.

```
kubectl apply -f manifests/20-fixes.yaml
```

```
deployment.apps/depot-api configured
deployment.apps/shipment-worker configured
deployment.apps/manifest-indexer configured
service/depot-tracker configured
```

Note configured, not created - these are updates to the objects you

already applied. Note also that the depot-tracker Deployment is absent

from the fix file: it was never broken. Only its Service was.

Step 7.2 - Wait ~60 seconds and confirm all five phases pass everywhere.

```
kubectl get pods -n kcna-lab25 -o wide
```

NAME	NOMINATED NODE	READY READINESS GATES	STATUS	RESTARTS	AGE	IP	NODE
depot-api-6b8d5f9c74-t7prv		1/1	Running	0	52s	10.244.0.27	kind-
control-plane	<none>	<none>					
depot-portal-6f9c8d7b54-4nvqz		1/1	Running	0	22m	10.244.0.21	kind-
control-plane	<none>	<none>					
depot-portal-6f9c8d7b54-mzt8h		1/1	Running	0	22m	10.244.0.22	kind-
control-plane	<none>	<none>					
depot-tracker-5c8f7b9d64-h6wnr		1/1	Running	0	18m	10.244.0.25	kind-
control-plane	<none>	<none>					
depot-tracker-5c8f7b9d64-q4jsx		1/1	Running	0	18m	10.244.0.26	kind-
control-plane	<none>	<none>					
manifest-indexer-7c9f8b6d54-k2xnp		1/1	Running	0	52s	10.244.0.28	kind-
control-plane	<none>	<none>					
shipment-worker-6d5c9b8f47-r3mqz		1/1	Running	0	52s	10.244.0.29	kind-
control-plane	<none>	<none>					
trriage-client		1/1	Running	0	22m	10.244.0.20	kind-
control-plane	<none>	<none>					

No Pending, no ImagePullBackOff, no CrashLoopBackOff, zero restarts.

Step 7.3 - Confirm DEP-1044 specifically. The endpoints must now appear.

```
kubectl get endpointslices -n kcna-lab25 -l kubernetes.io/service-name=depot-tracker
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-tracker-9xz4k	IPv4	5678	10.244.0.25,10.244.0.26	19m

```
kubectl exec -n kcna-lab25 triage-client -- \
  wget -q0- --timeout=5 http://depot-tracker.kcna-lab25.svc.cluster.local/
```

```
depot-tracker ok
```

Step 7.4 - Confirm DEP-1042's worker is now doing real work.

```
kubectl logs -n kcna-lab25 -l app=shipment-worker --tail=3
```

```
[worker] starting shipment worker for depot sg-tuas
[worker] 07:02:14 processed shipment batch
[worker] 07:02:29 processed shipment batch
```

Step 7.5 - Complete every column of data/triage-worksheet.csv. This is your assessment artefact.

Part 8 - Failure injection (mandatory): work one blind

Everything so far came with a ticket telling you which workload to look at. In the assessment it will not. Practise the method with no starting point.

Step 8.1 - Re-break exactly one thing, chosen at random by the shell so you do not know which:

```
CHOICE=$(( (RANDOM % 2) + 1 ))
case $CHOICE in
  1) kubectl set image deployment/depot-api api=nginx:1.27-alpine -n kcna-lab25 ;;
  2) kubectl patch service depot-tracker -n kcna-lab25 \
    -p '{"spec":{"selector":{"app":"depot-trackr"}}}' ;;
esac
echo "injected - do not scroll up"
```

```
deployment.apps/depot-api image updated
injected - do not scroll up
```

Step 8.2 - Find it using only the method. Start wide:

```
kubectl get pods -n kcna-lab25 -o wide
kubectl get endpointslices -n kcna-lab25
```

Work Q1 → Q5. Do not guess. Whichever question first answers NO tells you which of the two injections you received.

Step 8.3 - Time yourself. A competent engineer localises either of these to a phase in under 60 seconds with two commands.

Step 8.4 - Repair, whichever it was:

```
kubectl apply -f manifests/20-fixes.yaml
kubectl rollout status deployment/depot-api -n kcna-lab25 --timeout=120s
```

```
deployment.apps/depot-api configured  
service/depot-tracker configured  
deployment "depot-api" successfully rolled out
```

Step 8.5 - Verify the repair with the checks script.

```
bash verification/checks.sh
```

6. Verification

```
bash verification/checks.sh
```

The script verifies the healthy end state and, importantly, that the four broken manifests are still schema-valid - see [verification/expected-output.md](#).

You have completed the lab when all checks pass, [data/triage-worksheet.csv](#) is fully filled in, and you can reproduce the five-question decision tree from memory.

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix	
Pending, NODE <none>, FailedScheduling: Insufficient cpu/memory	Requests exceed any node's allocatable capacity	<code>`kubectl describe node <node> \</code>	<code>sed -n '/Allocated resources/,/^Events/p`</code>	Lower resources.requests, or add capacity
Pending with didn't match Pod's node affinity/selector	nodeSelector/nodeAffinity matches no node's labels	<code>kubectl get nodes --show-labels</code>	Correct the selector, or label the node	
Pending with untolerated taint	Node is tainted; Pod has no matching toleration	<code>`kubectl describe node <node> \</code>	<code>grep -i taint`</code>	Add a toleration, or remove the taint (with care)
Pending and no FailedScheduling event at all	The scheduler is not running - nothing is evaluating the Pod	<code>kubectl -n kube-system get pods -l component=kube-scheduler</code>	Escalate to a cluster administrator. Do not attempt on a shared cluster	
ErrImagePull / ImagePullBackOff with not found	Wrong image name or tag	<code>`kubectl describe pod <pod> \</code>	<code>tail -12`</code>	Correct the reference; compare with a known-good workload
ImagePullBackOff with unauthorized	Private registry, no credentials	Same	Create an imagePullSecret and set imagePullSecrets	
ImagePullBackOff with toomanyrequests	Registry rate limit	Same	Authenticate to the registry, or use a pull-through cache	
CrashLoopBackOff, exit code 127	command/args reference a binary absent from the image	<code>kubectl logs <pod> --previous</code>	Fix the command, or use an image that contains the binary	
CrashLoopBackOff, exit code 1	Application-level error	<code>kubectl logs <pod> --previous</code>	Read the log; usually configuration	
CrashLoopBackOff, exit code 137, reason: OOMKilled	Memory limit too low	<code>kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'</code>	Raise limits.memory, or fix the leak	
CrashLoopBackOff, exit code 137, reason: Error + Unhealthy events	Liveness probe killing a healthy-but-slow app	<code>kubectl describe pod <pod></code>	Add a startupProbe (Lab 22)	
0/1 Running, RESTARTS 0, never Ready	Readiness probe failing	<code>`kubectl describe pod <pod> \</code>	<code>grep -A3 Readiness`</code>	Fix the probe path/port (Lab 22)
Pods Ready but Service ENDPOINTS <unset>	Service selector matches no Pod labels	<code>kubectl get pods -l <the service's selector> returns nothing</code>	Align spec.selector with the Pod template labels	
Endpoints present but PORTS <unset>	targetPort names a port the container does not declare	<code>kubectl get deploy <d> -o jsonpath='{.spec.template.spec.containers[0].ports}'</code>	Name the container port, or use the numeric port	

Symptom	Likely cause	Diagnostic command	Fix	
Service resolves but times out (rather than refusing)	NetworkPolicy, or the app is not listening on that port	kubectl get networkpolicy -n <ns>	Adjust the policy, or correct the listen port	
kubectl logs says "waiting to start"	You are asking for logs before the container exists - a Q1/Q2 failure	kubectl get pod <pod> -o wide	Go back to Q1; use describe, not logs	
kubectl get componentstatuses errors or is empty	Deprecated in v1.19+ and unreliable on modern clusters	kubectl get --raw='/readyz?verbose'	Not a fault - use readyz/livez	

8. Cleanup

Delete only this lab's namespace.

```
kubectl delete namespace kcna-lab25
```

```
namespace "kcna-lab25" deleted
```

```
kubectl get namespace kcna-lab25
```

```
Error from server (NotFound): namespaces "kcna-lab25" not found
```

This lab created no cluster-scoped objects and modified nothing in

kube-system - Part 6 was entirely read-only. Deleting the namespace is a

complete cleanup.

Keep your completed data/triage-worksheet.csv. It is the assessment artefact.

9. What you learned

* Phase before cause. Five questions - scheduled, pulled, started, ready, endpoints - localise any workload failure in about two commands. Stop at the first NO. You applied four broken manifests and localised all four phases from a single `kubectl get pods -o wide`.

* Schema-valid is not working. All four broken manifests were accepted by the API server without a murmur. Validation checks structure; it cannot check that a tag is spelled right, a binary exists, a node has 512Gi, or a selector matches anything.

* Match the tool to the phase. `describe/events` for scheduler and kubelet decisions; `logs --previous` only once a container has actually run; `EndpointSlice` inspection for routing. Asking for logs on a Pending Pod returns a misleading answer.

* Exit codes are evidence. 127 = command not found (packaging); 1 = application error; 137 = SIGKILL, so check reason for OOMKilled; 0 = clean exit.

* The scheduler decides on requests, not usage. A near-idle node will still refuse a Pod whose requests do not fit.

* Service selector mismatches are silent. No event, no restart, no log - only absent endpoints. `kubectl get pods -l <service selector>` is the one-command test.

* Connection refused and timeout mean different things. Refused points at an endpoint-less Service; timeout points at NetworkPolicy or a wrong listen port.

* Control-plane components are static Pods run by the kubelet straight from /etc/kubernetes/manifests/, with no controller, no validation and no undo. Diagnose them with /readyz?verbose and kube-system logs; componentstatuses is deprecated. Read, never write.

* A Pending Pod with no FailedScheduling event at all is a scheduler outage, not an unschedulable Pod. Different problem, different escalation.

Further reading

* Kubernetes - *Troubleshooting Applications*: <<https://kubernetes.io/docs/tasks/debug/debug-application/>>

* Kubernetes - *Debug Running Pods*:
<<https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>>

* Kubernetes - *Debug Services*: <<https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>>

* Kubernetes - *Troubleshooting Clusters*: <<https://kubernetes.io/docs/tasks/debug/debug-cluster/>>

* Kubernetes - *Static Pods*: <<https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/>>

* Kubernetes - *Node-pressure Eviction*:
<<https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>>

* Kubernetes - *EndpointSlices*: <<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>>

* CNCF - KCNA curriculum, *Kubernetes Fundamentals* and *Cloud Native Architecture*:
<<https://github.com/cncf/curriculum>>